5 Gestione delle persone e organizzazione del team

OBIETTIVI FORMATIVI

- Essere consapevoli dell'importanza dei problemi delle persone nello sviluppo del software
- Conoscere diversi modi di organizzare il lavoro
- Conoscere i principali tipi di stili di gestione
- Apprezzare i diversi modi di organizzare un team di sviluppo software

Trovare la giusta struttura organizzativa e il giusto mix di competenze per un team di sviluppo è una questione difficile. Una teoria ben fondata non è prontamente disponibile per questo. Tuttavia, molte storie di progetti di successo e meno riusciti illustrano alcune delle complessità dei problemi del team di progetto. Questo capitolo delinea le principali questioni coinvolte.

- Le persone sono la risorsa più importante dell'organizzazione.
  Humphrey (**1997a**)

Nella maggior parte delle organizzazioni che sviluppano software, programmatori, analisti e altri professionisti lavorano insieme in un team. Un'adeguata struttura del team dipende da molti fattori, come il numero di persone coinvolte, la loro esperienza e coinvolgimento nel progetto, il tipo di progetto, le differenze e lo stile individuali. Questi fattori influenzano anche il modo in cui i progetti vengono gestiti. In questo capitolo, discutiamo vari aspetti della gestione delle persone, nonché alcune delle organizzazioni di team più comuni per i progetti di sviluppo software.

Il lavoro da svolgere nell'ambito di un progetto, sia esso un progetto di sviluppo software, la costruzione di una casa o la progettazione di una nuova automobile, comporta una serie di compiti. Una parte fondamentale della responsabilità di gestione è coordinare i compiti di tutti i partecipanti.

Questo coordinamento può essere svolto in diversi modi. Ci sono influenze sia esterne che interne sul meccanismo di coordinamento. Le influenze interne derivano dalle caratteristiche del progetto. Le influenze esterne provengono dall'ambiente organizzativo del progetto. Se queste influenze richiedono meccanismi di coordinamento contrastanti, i conflitti tra il progetto e l'ambiente sono in agguato dietro l'angolo.

Si consideri, ad esempio, un progetto di sviluppo software altamente innovativo da realizzare all'interno di un ente governativo. Le caratteristiche del progetto possono richiedere un meccanismo di coordinamento di tipo flessibile e informale, in cui l'impegno di persone specializzate, piuttosto che una stretta aderenza alle procedure formali, è un fattore critico di successo. D'altra parte, l'ambiente può essere orientato verso una burocrazia a controllo centralizzato, che cerca di imporre procedure formali alla gestione dei progetti. Questi due meccanismi non funzionano armoniosamente insieme. Di conseguenza, la gestione del progetto potrebbe essere schiacciata tra quelle forze opposte.

La sezione 5.1 elabora ulteriormente i vari fattori interni ed esterni che influenzano il modo in cui i progetti vengono gestiti e sottolinea la necessità di prestare ampia attenzione all'elemento umano nella gestione dei progetti.

Lo sviluppo del software implica il lavoro di squadra. I membri del team devono coordinare il proprio lavoro, comunicare le proprie decisioni, ecc. Per un piccolo progetto, il team sarà composto da un massimo di poche persone. All'aumentare delle dimensioni del progetto, aumenta anche la dimensione del team. Tuttavia, le grandi squadre sono difficili da gestire. Coordinare il lavoro di una grande squadra è difficile. La comunicazione tra i membri del team tende ad aumentare in modo esponenziale con le dimensioni del team (vedi anche il Capitolo 7). Pertanto, le squadre grandi sono solitamente suddivise in squadre più piccole in un modo che limita la maggior parte del coordinamento e della comunicazione all'interno del sotto-team.

La sezione 5.2 discute diversi modi per organizzare un team di sviluppo software. Le organizzazioni gerarchiche e a matrice possono essere trovate anche in altri tipi di attività, mentre il capo programmatore, SWAT e il team agile sono specifici per lo sviluppo del software. Sebbene i progetti open source non abbiano mezzi per imporre la struttura del team, di solito convergono in un'organizzazione simile a una cipolla, come discusso nella Sezione 5.2.6 .

A causa dell'outsourcing, delle società in rete e della globalizzazione, lo sviluppo del software è diventato un'attività distribuita. Le squadre, ad esempio, di Amsterdam, Boston e Bangalore potrebbero dover collaborare allo sviluppo dello stesso sistema. Come dovremmo dividere i compiti tra questi gruppi? Come possiamo garantire che la comunicazione tra questi gruppi sia efficace? Anche le differenze culturali svolgono un ruolo nello sviluppo di più siti. Ad esempio, le persone in Asia rispettano l'autorità. In Nord America, è più consuetudine che i membri del team litigano con il proprio manager. I manager e i membri del team dovrebbero essere consapevoli di tali differenze e agire di conseguenza. I problemi relativi alle persone che influiscono sullo sviluppo del software multisito sono discussi nel Capitolo 20 .

5.1 GESTIONE DELLE PERSONE

Una squadra è composta da individui, ognuno dei quali ha obiettivi personali. È compito della gestione del progetto escludere una squadra da questi individui, per cui gli obiettivi individuali sono riconciliati in un obiettivo per il progetto nel suo insieme.

Sebbene gli obiettivi individuali delle persone possano differire, è importante identificare gli obiettivi del progetto in una fase iniziale e comunicarli inequivocabilmente ai membri del progetto. I membri del progetto dovrebbero sapere cosa ci si aspetta da loro. Se c'è qualche incertezza al riguardo, i membri del team determineranno i propri obiettivi: un programmatore può decidere che l'efficienza ha la massima priorità, un altro può scegliere un uso efficiente della memoria, mentre ancora un terzo deciderà che scrivere molto codice è ciò che conta. Obiettivi così ampiamente divergenti possono portare a gravi problemi.

Una volta che gli obiettivi del progetto sono stati stabiliti e il progetto è in corso, è necessario monitorare e valutare le prestazioni dei membri del progetto rispetto agli obiettivi del progetto. Questo può essere difficile, poiché gran parte di ciò che viene fatto è invisibile e il progresso è difficile da misurare.

Idealmente, vorremmo avere un'indicazione della funzionalità fornita e definire la produttività come la quantità di funzionalità fornita per unità di tempo. La produttività è principalmente definita come il numero di righe di codice consegnate per mese-uomo. Tutti saranno d'accordo sul fatto che questa misura non è ottimale, ma non è stato trovato niente di meglio. Uno dei grandi pericoli dell'utilizzo di questa misura è che le persone tendono a produrre quanto più codice possibile. Questo ha un effetto molto dannoso. Il fattore di costo più importante nei progetti di sviluppo software è la quantità di codice da fornire (vedi anche **il Capitolo 7**): scrivere meno codice è più economico. Il riutilizzo del codice esistente è un modo per risparmiare tempo e denaro; dovrebbe, quindi, essere fortemente raccomandato. L'utilizzo della quantità di codice fornita per mese-uomo come indicatore di produttività non crea alcun incentivo per il riutilizzo del software.

Un altro aspetto della valutazione delle persone si verifica nei processi di gruppo come le revisioni tra pari, le ispezioni e le procedure dettagliate. Queste tecniche vengono utilizzate durante le attività di verifica e validazione, per scoprire errori o per valutare la qualità del codice o della documentazione. Per rendere efficaci questi processi è necessario separare nettamente i documenti da valutare dai loro autori. Weinberg (**1971**) ha usato il termine "programmazione senza ego" in questo contesto. Una valutazione del prodotto del lavoro di qualcuno non dovrebbe implicare una valutazione di quella persona.

Uno dei maggiori problemi nello sviluppo del software è il coordinamento delle attività dei membri del team. Man mano che i progetti di sviluppo crescono e diventano più complessi, i problemi di coordinamento si accumulano rapidamente. Per contrastare questi problemi, la direzione formalizza la comunicazione, ad esempio organizzando riunioni formali di progetto, ispezioni rigorosamente monitorate e un consiglio ufficiale di controllo della configurazione. Tuttavia, la comunicazione interpersonale informale è nota per essere il modo principale in cui le informazioni fluiscono all'interno e attraverso un'organizzazione di sviluppo. Non è saggio escludere del tutto questo tipo di comunicazione. [1] La comunicazione interpersonale informale è più facile se le persone sono fisicamente a stretto contatto. Ancora peggio, le persone sono inclini a barattare la facilità con cui le informazioni possono essere ottenute con la loro qualità. Accetteranno il consiglio del vicino, anche se sanno che si possono trovare consigli molto migliori al piano successivo. Per contrastare questa tendenza, è opportuno riunire diversi stakeholder in modi controllati, ad esempio disponendo di esperti di dominio nel team di progettazione, coinvolgendo utenti nel test del software o attraverso approcci di progettazione partecipata. La collocazione di tutte le parti interessate è un aspetto principale dei team agili.

I team di sviluppo software di successo mostrano un mix di qualità: competenza tecnica, empatia dell'utente finale e consapevolezza organizzativa. La competenza tecnica è, ovviamente, richiesta in primo luogo per fornire un sistema di alta qualità. L'empatia dell'utente finale e la consapevolezza organizzativa hanno a che fare con il riconoscimento degli individui e dell'organizzazione che devono far fronte al sistema. Una combinazione di questi orientamenti in un team aiuta a garantire che venga prestata sufficiente attenzione a ciascuno degli aspetti (Klein *et al.*, **2002**).

La gestione del team comporta moltissimi aspetti, non ultimo dei quali riguarda la cura dell'elemento umano. I successi tra i progetti di sviluppo software possono spesso essere ricondotti a una forte attenzione alle preoccupazioni culturali e sociologiche, come gli sforzi per creare una cultura senza colpa o la sollecitazione all'impegno e alla partnership. Questo capitolo tocca solo alcuni aspetti di questo. Brooks (**1995**) e DeMarco e Lister (**1999**) forniscono molte osservazioni perspicaci sull'elemento umano della gestione dei progetti software.

Nel resto di questa sezione, ci limitiamo a due tassonomie piuttosto generali di meccanismi di coordinamento e stili di gestione.

5.1.1 Meccanismi di coordinamento

Nel suo testo classico, Mintzberg (1983) distingue tra cinque tipiche configurazioni organizzative. Ogni configurazione riflette un ambiente ideale ed è associata a un meccanismo di coordinamento specifico: un meccanismo preferito per coordinare i compiti da svolgere all'interno di quel tipo di configurazione. Le configurazioni di Mintzberg e i meccanismi di coordinamento associati sono i seguenti:

- **Struttura semplice** In una struttura semplice, possono esserci uno o pochi manager e un nucleo di persone che svolgono il lavoro. Il corrispondente meccanismo di coordinamento è chiamato *supervisione diretta.* Questa configurazione si trova spesso in organizzazioni nuove e relativamente piccole. C'è poca specializzazione, formazione e formalizzazione. Il coordinamento spetta a persone separate, che sono responsabili del lavoro degli altri.
- **Burocrazia maccanica** Quando il contenuto del lavoro è completamente specificato, diventa possibile eseguire e valutare i compiti sulla base di istruzioni precise. La produzione in serie e le linee di montaggio sono esempi tipici di questo tipo di configurazione. C'è poca formazione e molta specializzazione e formalizzazione. Il coordinamento si ottiene attraverso *la standardizzazione dei processi di lavoro.*
- **Soluzione divisionale** In questo tipo di configurazione, ad ogni divisione (o progetto) è concessa una notevole autonomia su come raggiungere gli obiettivi dichiarati. I dettagli operativi sono lasciati alla divisione stessa. Il coordinamento si ottiene attraverso *la standardizzazione dei risultati del lavoro.* Il controllo viene eseguito misurando regolarmente le prestazioni della divisione. Questo meccanismo di coordinamento è possibile solo quando il risultato finale è specificato con precisione.

- **Burocrazia professionale** Se non è possibile specificare né il risultato finale né i contenuti del lavoro, il coordinamento può essere raggiunto attraverso *la standardizzazione delle competenze dei lavoratori.* In una burocrazia professionale, ai professionisti qualificati viene concessa una notevole libertà su come svolgere il proprio lavoro. Gli ospedali sono esempi tipici di questo tipo di configurazione.
- **Adhocracy** Nei progetti di grandi dimensioni o di natura innovativa, il lavoro è suddiviso tra molti specialisti. Potremmo non essere in grado di dire esattamente cosa dovrebbe fare ogni specialista o come dovrebbe svolgere i compiti loro assegnati. Il successo del progetto dipende dalla capacità del gruppo nel suo insieme di raggiungere un obiettivo non specificato in un modo non specificato. Il coordinamento si ottiene attraverso *l'adeguamento reciproco.*

I meccanismi di coordinamento distinti da Mintzberg corrispondono a configurazioni organizzative tipiche, come un ospedale o una fabbrica a catena di montaggio. Secondo Mintzberg, diverse organizzazioni richiedono meccanismi di coordinamento diversi. Le organizzazioni non sono tutte uguali. Seguendo questa linea di pensiero, è probabile che fattori esterni a un progetto di sviluppo software esercitino un'influenza sui meccanismi di coordinamento di quel progetto.

Si noti che la maggior parte delle organizzazioni reali non si adatta a un singolo tipo di configurazione. Parti diverse di un'organizzazione possono essere organizzate in modo diverso. Inoltre, le configurazioni di Mintzberg rappresentano ideali astratti. In realtà, le organizzazioni possono tendere verso una di queste configurazioni, ma portare anche aspetti di altre.

5.1.2 Stili di gestione

Lo sviluppo di un sistema software, la costruzione di una casa, la progettazione e la partecipazione a una vacanza in famiglia sono paragonabili in quanto riguardano ciascuno uno sforzo coordinato svolto da un gruppo di persone. Sebbene sia probabile che questi progetti vengano trattati in modi molto diversi, i presupposti di base che stanno alla base delle loro strutture organizzative e stili di gestione hanno molto in comune. Questi presupposti di base possono essere evidenziati distinguendo tra due dimensioni nella gestione delle persone:

- **Direzionalità relazionale** Riguarda l'attenzione verso un individuo e la sua relazione con gli altri individui all'interno dell'organizzazione.
- **Orientamento dei compiti** Riguarda l'attenzione ai risultati da raggiungere e al modo in cui questi risultati devono essere raggiunti.

Sia la relazione che la direzione del compito possono essere alte o basse. Questo porta a quattro combinazioni di base, come illustrato nella **Figura 5.1**. Ovviamente, queste combinazioni corrispondono a orientamenti estremi. Per ogni dimensione, c'è un intero spettro di possibilità.

		Task directedness: low	Task directedness: high
Relation directedness	low	separation style	commitment style
	high	relation style	integration style

Figura 5.1Quattro stili di gestione di base, dopo (Reddin, 1970)

Lo stile più appropriato per una determinata situazione dipende dal tipo di lavoro da svolgere:

- **separazione** Questo stile di gestione è generalmente più efficace per il lavoro di routine. L'efficienza è il tema centrale. La direzione agisce come un burocrate e applica regole e procedure. Il lavoro è coordinato gerarchicamente. Il processo decisionale è dall'alto verso il basso, formale e basato sull'autorità. Uno dei principali vantaggi di questo stile è che si traduce in un'organizzazione del progetto stabile. D'altra parte, le vere innovazioni sono difficili da realizzare. Questo stile corrisponde strettamente al coordinamento di Mintzberg attraverso la standardizzazione dei processi di lavoro.
- **relazione** Questo stile è solitamente più efficace in situazioni in cui le persone devono essere motivate, coordinate e formate. I compiti da svolgere sono vincolati alle persone. Il lavoro non ha carattere di routine, ma innovativo, complesso e specializzato. Il processo decisionale è un processo di gruppo; implica la negoziazione e la costruzione del consenso. Un evidente punto debole di questo stile è che può portare a infiniti incontri caotici. La capacità del manager di moderare un processo decisionale efficiente è un fattore chiave di successo. Questo stile si adatta meglio al meccanismo di coordinamento dell'adattamento reciproco di Mintzberg.
- **Stile di impegno** Questo è più efficace se il lavoro viene svolto sotto pressione. Perché questo stile sia efficace, il manager deve sapere come raggiungere gli obiettivi senza suscitare risentimento. Il processo decisionale non avviene durante le riunioni. Piuttosto, le decisioni sono implicate dalla visione condivisa del team degli obiettivi del progetto. Un potenziale punto debole di questo stile è che, una volta che questa visione è stata concordata, il team non risponde ai cambiamenti nel suo ambiente, ma inciampa alla cieca lungo la strada tracciata. Questo stile si adatta meglio alla burocrazia professionale di Mintzberg.
- **Stile di integrazione** Si adatta a situazioni in cui il risultato è incerto. Il lavoro è di natura esplorativa ei vari compiti sono fortemente interdipendenti. È compito del manager stimolare e motivare. Il processo decisionale è informale, dal basso verso l'alto. Questo stile promuove la creatività e le persone sono sfidate a ottenere il meglio da se stesse. Un possibile punto debole di questo stile è che gli obiettivi dei singoli membri del team vengono disconnessi da quelli del progetto e i membri del team iniziano a competere tra loro. Ancora una volta, il coordinamento di Mintzberg attraverso l'adattamento reciproco si adatta bene a questa situazione.

Ciascuno dei meccanismi di coordinamento e degli stili di gestione individuati può essere utilizzato all'interno di progetti di sviluppo software. È solo ragionevole aspettarsi che progetti con caratteristiche ampiamente diverse richiedano meccanismi diversi. Un team esperto a cui è stato chiesto di sviluppare un'applicazione ben specificata in un dominio familiare può ottenere il coordinamento attraverso la standardizzazione dei processi di lavoro. Tuttavia, per un'applicazione complessa e innovativa, è improbabile che questo meccanismo funzioni.

Nel Capitolo 8 , identifichiamo vari tipi di progetti di sviluppo software e indichiamo quale tipo di meccanismo di coordinamento e stile di gestione si adatta meglio a tali progetti. Si noti che i meccanismi di coordinamento suggeriti nel capitolo 8 derivano da fattori interni, ovvero caratteristiche del progetto in esame. Come notato in precedenza, l'ambiente del progetto influenzerà anche la sua organizzazione.

Si noti che nella discussione di cui sopra abbiamo guardato dal manager al team e ai suoi membri. In alternativa, possiamo esaminare la relazione e la *maturità dei compiti* dei singoli membri del team. La maturità della relazione riguarda l'atteggiamento dei dipendenti nei confronti del proprio lavoro e della propria gestione. La maturità del compito riguarda la competenza tecnica. È importante che il manager allinei i suoi rapporti con i membri del team con le rispettive relazioni e maturità del compito. Ad esempio, un neolaureato può avere un'elevata maturità nei compiti e una bassa maturità relazionale, quindi la sua introduzione in un team qualificato può giustificare un'attenta guida.

5.2 ORGANIZZAZIONE DEL TEAM

All'interno di una squadra si possono distinguere diversi ruoli. Ci sono manager, tester, designer, programmatori e così via. A seconda delle dimensioni del progetto, più di un ruolo può essere svolto da una persona o persone diverse possono svolgere lo stesso ruolo. Le responsabilità e i compiti di ciascuno di questi ruoli devono essere definiti con precisione nel piano di progetto.

Le persone collaborano all'interno di un team per ottenere un risultato ottimale. Tuttavia è consigliabile separare rigorosamente alcuni ruoli. È una buona idea creare un team di test indipendente dal team di sviluppo. Allo stesso modo, la garanzia della qualità dovrebbe, in linea di principio, essere condotta da persone non direttamente coinvolte nel processo di sviluppo.

Le squadre grandi sono difficili da gestire e quindi sono spesso divise in squadre più piccole. Definendo chiaramente i compiti e le responsabilità dei vari sottogruppi, la comunicazione può essere in gran parte confinata alla comunicazione tra i membri dello stesso sottogruppo. La quantificazione del costo della comunicazione interpersonale fornisce informazioni sull'effetto delle dimensioni del team sulla produttività e aiuta a strutturare in modo efficace grandi team di sviluppo. Alcune semplici formule per farlo sono derivate nel **Capitolo 7**.

Nelle seguenti sottosezioni discutiamo diverse forme organizzative per i team di sviluppo software.

5.2.1 Organizzazione gerarchica

In un ambiente completamente dedicato alla produzione di software, spesso incontriamo strutture gerarchiche di team. A seconda delle dimensioni dell'organizzazione o del progetto, si possono distinguere diversi livelli di gestione.

La Figura 5.2 fornisce un esempio di organizzazione gerarchica. I rettangoli denotano i vari **sottogruppi** in cui viene svolto il lavoro vero e proprio. I nodi cerchiati denotano i **gestori**. In questo esempio si possono distinguere due livelli di gestione. Al livello inferiore, diversi team sono responsabili delle diverse parti del progetto. I manager a questo livello hanno una responsabilità primaria nel coordinare il lavoro all'interno dei rispettivi team. Al livello superiore, il lavoro delle diverse squadre è coordinato.

Questo tipo di organizzazione gerarchica riflette spesso la struttura globale del sistema da sviluppare. Se il sistema ha tre sottosistemi principali, ci possono essere tre squadre, una per ogni sottosistema, come nella **Figura 5.2**. Potrebbero esserci anche unità funzionali associate a responsabilità specifiche a livello di progetto, come la garanzia della qualità e i test.

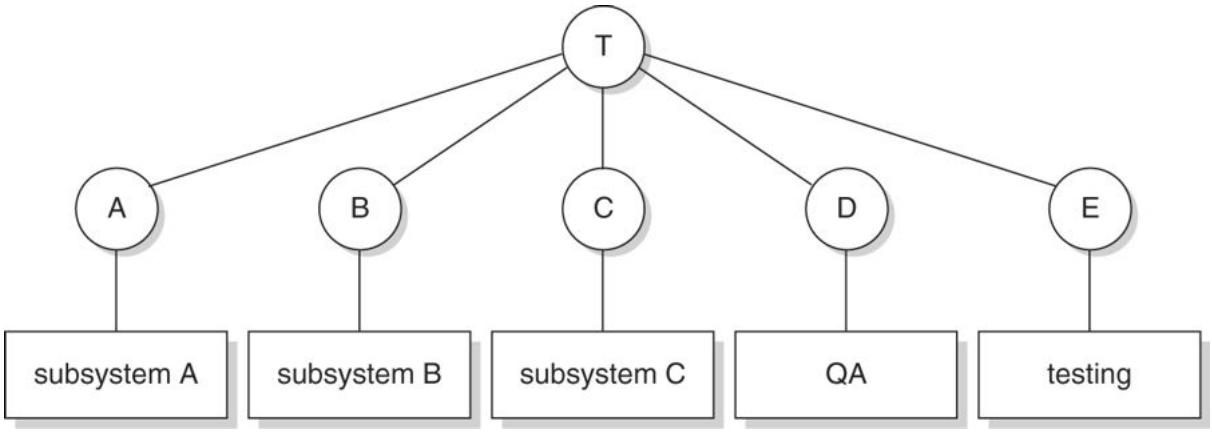


Figura 5.2Un'organizzazione gerarchica del team

Non è possibile associare l'organizzazione gerarchica ad uno solo dei meccanismi di coordinamento sopra introdotti. Per ciascuna unità individuata è possibile uno qualsiasi dei meccanismi di coordinamento prima citati. Inoltre, non è necessario applicare lo stesso meccanismo in ogni nodo della gerarchia. Tuttavia, avere diversi meccanismi di coordinamento all'interno di un progetto non è privo di problemi.

Sulla base di un'analisi delle caratteristiche dei vari sottosistemi, i rispettivi gestori potrebbero voler scegliere uno stile di gestione e un meccanismo di coordinamento che meglio si adatti a tali caratteristiche. Qualora uno o più dei sottosistemi siano di natura fortemente innovativa, il management può optare per un tipo di coordinamento di adeguamento

reciproco. I livelli più alti all'interno di una gerarchia tenderanno solitamente a un meccanismo di coordinamento basato su una qualche forma di standardizzazione, imponendo regole e procedure come in una burocrazia macchina o misurando l'output come in una configurazione divisionalizzata. In tali casi, poteri interni ed esterni possono benissimo scontrarsi a uno o più livelli intermedi.

Un altro punto critico in qualsiasi organizzazione gerarchica è la **distanza** tra la parte superiore e quella inferiore della piramide gerarchica. Il lavoro "reale" è generalmente svolto ai livelli più bassi di questa piramide. Le persone a questi livelli inferiori possiedono generalmente la vera conoscenza dell'applicazione. Più si sale nella gerarchia, meno specifica diventa la conoscenza (questo è il motivo principale per cui la gestione a questi livelli superiori tende al coordinamento attraverso la standardizzazione). Tuttavia, la maggior parte delle decisioni viene presa a un livello abbastanza alto. In molti casi, i segnali dal livello inferiore vengono in qualche modo sussunti in uno dei livelli intermedi.

Man mano che le informazioni filtrano attraverso i vari livelli della gerarchia, tendono a diventare sempre più color rosa. Il seguente scenario non è del tutto fittizio:

- in basso: abbiamo gravi problemi nell'implementazione del modulo X;
- livello 1: ci sono dei problemi con il modulo X;
- livello 2: il progresso è costante, non prevedo problemi reali;
- in alto: tutto procede secondo il nostro piano.

Questi tipi di distorsione sono difficili da aggirare del tutto. Sono, tuttavia, rafforzati dal fatto che la linea organizzativa lungo la quale vengono riportati i progressi è anche la linea lungo la quale vengono misurate e valutate le prestazioni dei membri del team. Ognuno è favorito da una valutazione positiva ed è quindi propenso a colorare i report di conseguenza. Se i dati sullo stato di avanzamento di un progetto vengono raccolti ed elaborati da persone non direttamente coinvolte nella valutazione dei membri del team, è molto più probabile che le informazioni raccolte siano di sufficiente affidabilità.

Un aspetto altrettanto problematico delle organizzazioni gerarchiche risiede nel fatto che si viene giudicati, sia socialmente che finanziariamente, in base al *livello* in cui ci si trova all'interno dell'organizzazione. È quindi naturale aspirare a livelli sempre più alti all'interno della gerarchia. Tuttavia, non è affatto chiaro se ciò sia auspicabile. Secondo il principio di Peter, in un'organizzazione gerarchica ogni dipendente sale al livello della sua incompetenza (Peter e Hull, **1970**). Un buon programmatore non ha bisogno di essere un buon manager. Un buon programmatore richiede determinate abilità e un buon manager richiede abilità diverse. A lungo termine, sembra più saggio mantenere le persone a un livello in cui si comportano bene e premiarle di conseguenza.

5.2.2 Organizzazione a matrice

In un ambiente in cui il software è un semplice sottoprodotto, spesso incontriamo una sorta di organizzazione a matrice. Le persone di diversi dipartimenti vengono quindi assegnate a un progetto di sviluppo software, possibilmente part-time. In questo tipo di organizzazione, a volte è difficile controllare i progressi. Un dipendente deve soddisfare diversi capi e può avere la tendenza a mettere un capo contro un altro.

Possiamo anche utilizzare un'organizzazione a matrice in un ambiente completamente dedicato allo sviluppo del software. L'unità di base è un piccolo gruppo specializzato. Potrebbero esserci più unità con la stessa specializzazione. Possibili specializzazioni sono, ad esempio, la programmazione grafica, i database, le interfacce utente e il controllo qualità. Le unità sono organizzate in base alla loro specialità. I progetti, invece, possono coinvolgere unità con specialità diverse. Le persone sono così organizzate lungo due assi, uno che rappresenta i vari gruppi di specialisti e uno che rappresenta i progetti a cui sono assegnati. Questo tipo di organizzazione matriciale è illustrato nella **Tabella 5.1**.

Tabella 5-1 Un'organizzazione matriciale

	Programmazione in tempo reale	Grafica	Data Base	QA	Test
Progetto C	X			X	X
Progetto B	X		X	X	X
Progetto A		X	X	X	X

In una tale situazione, il project manager è responsabile del buon fine del progetto. Il manager responsabile di una o più unità con la stessa specialità ha una missione a più lungo termine, come mantenere o ampliare le conoscenze e le competenze dei membri del suo team. Espresso in termini di dimensioni gestionali di base discusse in precedenza, è probabile che il project manager enfatizzi la direzionalità del compito, mentre il manager dell'unità enfatizzerà la direzionalità della relazione. Tale organizzazione può essere molto efficace, a condizione che vi sia sufficiente fiducia reciproca e disponibilità a cooperare e perseguire gli obiettivi del progetto.

5.2.3 Capo Squadra Programmatore

Il concetto di Capo Squadra Programmatore (in inglese Chief Programmer Team) è un modello organizzativo per lo sviluppo del software ideato dal pioniere dell'ingegneria del software Harlan Mills nel 1971. Il concetto è diventato celebre in tutto il mondo dopo essere stato

dettagliato da Fred Brooks nel suo saggio classico The Mythical Man-Month (dove viene metaforicamente paragonato a un'équipe chirurgica).

L'idea centrale di Mills era quella di strutturare i team di sviluppo intorno a un unico programmatore di eccezionale talento, riducendo drasticamente i problemi di comunicazione tipici dei grandi gruppi e massimizzando la produttività. Il **nucleo** di tale squadra è composto da tre persone:

- **Capo Programmatore (Chief Programmer / Il "Chirurgo"):** È l'elemento cardine. Possiede un talento superiore, oltre dieci anni di esperienza e una profonda conoscenza tecnica. Definisce personalmente le specifiche, progetta l'architettura, scrive il codice principale, esegue i test e redige la documentazione di base.
- **Copilota (Backup Programmer):** Un programmatore di alto livello, pronto a sostituire il capo in qualsiasi momento. Partecipa alla progettazione e alla stesura del codice, agendo come consulente interno e garantendo la continuità del progetto.
- **Segretario di Programmazione (Program Clerk / Librarian):** Un ruolo cruciale nell'era pre-Git. Gestisce l'archivio tecnico, registra le versioni del codice, i test eseguiti e mantiene la documentazione sia in formato cartaceo che digitale.

Oltre a queste tre persone, è possibile aggiungere un ulteriore (piccolo) numero di esperti al team di capo programmatori. Ci possono infatti essere altre figure secondarie come:

- **Amministratore (Administrator):** Gestisce gli aspetti burocratici, legali, finanziari e il personale. Il suo scopo è liberare il Capo Programmatore da qualsiasi distrazione manageriale.
- **Specialista dei Linguaggi (Language Lawyer):** L'esperto assoluto della sintassi e dei segreti del linguaggio di programmazione utilizzato, capace di trovare soluzioni ultra-efficienti per i problemi più complessi.
- **Creatore di Strumenti (Toolsmith):** Sviluppa utility personalizzate, macro e script per supportare l'attività del Capo Programmatore.
- **Collaudatore (Tester) ed Editore (Editor):** Figure dedicate rispettivamente alla creazione di casi di test avversariali e alla rifinitura formale della documentazione.

Perché nacque questo modello?

Harlan Mills notò che la produttività tra un programmatore eccellente e uno mediocre poteva variare anche di un ordine di grandezza (10:1). Invece di dividere un grande progetto tra tanti programmatori di medio livello (creando enormi colli di bottiglia comunicativi), Mills intuì che **era più efficiente mettere un solo "genio" nelle condizioni ideali per produrre il software**, circondandolo di un team di supporto.

Eredità moderna

Sebbene oggi non si utilizzi più la configurazione rigida del 1971, i principi del *Chief Programmer Team* sopravvivono in molte metodologie odierne:

- Il rapporto stretto tra Capo e Copilota ha anticipato il **Pair Programming** delle metodologie Agile.
- L'automazione moderna (sistemi di controllo versione come Git, pipeline CI/CD, IDE avanzati con IA) ha di fatto assorbito i ruoli del *Program Clerk* e del *Toolsmith*.

In questo tipo di organizzazione, il capo programmatore richiede requisiti piuttosto elevati. Il capo programmatore deve essere molto competente nell'area tecnica, ma deve anche avere sufficienti capacità di gestione. Ci sono abbastanza persone che possono svolgere il ruolo di capo programmatore? Possono sorgere problemi di competenza. Il capo programmatore gioca un ruolo molto centrale: prende tutte le decisioni; altri membri del team potrebbero sfidare alcune delle sue qualità.

L'idea iniziale di un team di capo programmatori sembra alquanto elitaria. Assomiglia a un team di chirurghi nella sua enfasi su compiti altamente specializzati e leadership carismatica. Tuttavia, i vantaggi di un team composto da un piccolo gruppo di pari rispetto a un enorme team di sviluppo che lotta per produrre sistemi software sempre più grandi possono essere riguadagnati in una forma modificata di capo del team di programmatori, in cui prevalgono gli aspetti del gruppo di pari. Il team di sviluppo è composto da un piccolo gruppo di persone responsabili collettivamente del compito in questione. In particolare, i lavori non sono strutturati intorno alle fasi del ciclo di vita. Non ci sono analisti, designer o programmatori, anche se il ruolo di tester può essere assegnato a una persona specifica. All'interno del gruppo possono verificarsi diversi livelli di competenza. Le persone più esperte agiscono come capo programmatore e vice capo programmatore. All'altra estremità della scala, uno o due tirocinanti possono essere assimilati e ottenere la necessaria formazione sul posto di lavoro. Un tirocinante può benissimo fungere da bibliotecario della squadra.

5.2.4 Squadra SWAT

Nei progetti con un modello di processo evolutivo o iterativo come Rapid Application Development (RAD), a volte viene utilizzata un'organizzazione di progetto nota come team Skilled With Advanced Tools (SWAT). In breve, è un gruppo temporaneo e interfunzionale di esperti di alto livello (architetti, sviluppatori, tester) riuniti per risolvere problemi tecnici critici e di alto livello o emergenze, come il salvataggio di progetti in difficoltà o la risoluzione di colli di bottiglia complessi e urgenti. Questi team si concentrano su interventi rapidi e specializzati, spesso utilizzando metodologie agili per ottenere risultati rapidi e di grande impatto.

Una squadra SWAT è relativamente piccola. In genere ha quattro o cinque membri. Preferibilmente, la squadra occupa una stanza. I canali di comunicazione sono mantenuti molto brevi. Il team non ha lunghe riunioni formali con verbali formali. Piuttosto, utilizza

workshop e sessioni di brainstorming di cui viene conservata poco più di un'istantanea di un disegno alla lavagna.

Un team SWAT in genere crea versioni incrementali di un sistema software. Per farlo in modo efficace, impiega componenti riutilizzabili, linguaggi di altissimo livello e potenti generatori di software. Il lavoro dei membri del team è supportato e coordinato tramite groupware o software di gestione del flusso di lavoro.

Come nel capo del team di programmatori, il leader di un team SWAT è come un caposquadra nel settore edile: è sia un manager che un collaboratore. I membri di una squadra SWAT sono generalisti. Possono avere determinate specialità, ma devono anche essere in grado di svolgere una serie di attività, come partecipare a un workshop con i clienti, costruire un prototipo e testare un software.

La motivazione della squadra è molto importante in una squadra SWAT. Una squadra SWAT adotta spesso un nome accattivante, un motto o un logo. Questa etichetta esprime quindi la loro visione. Gli individui traggono orgoglio e autostima dalla loro appartenenza a una squadra SWAT.

Pro e contro

Pro: Risoluzione rapida dei problemi, applicazione di conoscenze specialistiche e risoluzione dei problemi ad alto impatto.

Contro: Elevati costi delle risorse, potenziale interruzione dei team esistenti e rischio di diventare un sostegno permanente piuttosto che una soluzione temporanea.

5.2.5 Squadra Agile

Gli approcci agili allo sviluppo del software sono nati e hanno molto in comune con i vari approcci di sviluppo iterativo. Allo stesso modo, un team agile ha molto in comune, ad esempio, con un team SWAT: collocazione, canali di comunicazione brevi e un atteggiamento orientato alle persone piuttosto che formalistico. Spesso le persone lavorano in coppia, con pilota e copilota, ma senza una gerarchia.

Poiché i processi agili hanno poca disciplina applicata dall'esterno, hanno bisogno che la disciplina provenga dall'interno del team. I team agili hanno bisogno di autodisciplina. Se una coppia di programmatori sviluppa del codice e i test successivi falliscono, devono fare un passo indietro e rifare il proprio lavoro. Dopo aver incorporato un pezzo di lavoro, devono considerare il sistema nel suo insieme e, se necessario, refactoring.

Affinché ciò abbia successo, un team agile ha bisogno di persone più qualificate rispetto a un team che lavora secondo un approccio basato sulla pianificazione. In un approccio basato sulla pianificazione, il piano è come un giubbotto di salvataggio su cui le persone possono fare affidamento. In una squadra agile, non è disponibile un giubbotto di salvataggio del genere e le persone devono essere in grado di nuotare. In termini di livelli di comprensione discussi in

(Cockburn, **2002**), un team agile richiede persone che lavorano al livello 2 o 3 ed è considerato rischioso con persone che lavorano al livello 1 (vedi **Tabella 5.2**). Negli ambienti orientati alla pianificazione, le persone che lavorano ai livelli 2 o 3 sono richieste solo durante le fasi di definizione dello sviluppo. Successivamente, possono essere ospitate alcune persone che lavorano al livello 1.

Il team in SCRUM

Un team Scrum è organizzato in modo da favorire la collaborazione, la comunicazione e la flessibilità nello sviluppo di progetti, in particolare quelli legati al software. Si basa su tre ruoli principali e alcune pratiche chiave che permettono di rispettare il framework Agile. Ecco la struttura di base di un team Scrum.

Product Owner (PO): Rappresenta la voce del cliente o degli stakeholder e ha la responsabilità di definire le funzionalità e le priorità del prodotto. Gestisce il Product Backlog, un elenco di caratteristiche e funzionalità da sviluppare, e lavora a stretto contatto con il team per assicurare che il valore sia massimizzato per il prodotto finale.

Scrum Master: È il facilitatore del processo Scrum. Il suo compito è assicurarsi che il team segua le pratiche Scrum correttamente, risolva eventuali impedimenti che ostacolano il progresso del team e promuova una cultura collaborativa e autodisciplinata. Aiuta sia il team che il Product Owner a rimanere focalizzati sugli obiettivi.

Development Team (Team di Sviluppo): Composto da professionisti cross-funzionali che lavorano insieme per consegnare le funzionalità del prodotto. Il team ha la responsabilità di organizzare autonomamente il proprio lavoro durante lo Sprint (un ciclo di lavoro solitamente di 1-4 settimane) per raggiungere gli obiettivi definiti nel Sprint Backlog.

In questo modo, il team Scrum è auto-organizzato, con una chiara definizione dei ruoli e una forte enfasi sulla comunicazione continua e sul miglioramento iterativo.

5.2.6 Sviluppo di software open source

Uno dei primi libri sullo sviluppo di software open source è intitolato *The Cathedral and the Bazaar* (Raymond, **1999**). La cattedrale si riferisce allo sviluppo software tradizionale, pesante e gerarchico, come è comune nello sviluppo di software closed source. Al contrario, lo sviluppo di software open source è come un bazar: orde di sviluppatori anarchici sono casualmente organizzate in un'organizzazione virtuale in rete. La metafora del bazar è stata scelta per riflettere la forma balbettante, chiacchierata e apparentemente disorganizzata del mercato mediorientale. Sebbene la metafora del bazar possa adattarsi ad alcuni gruppi di sviluppo open source, molte comunità open source di successo hanno adottato la struttura più organizzata simile a una cipolla illustrata nella **Figura 5.3** .

Tabella 5.2 Livelli di comprensione

Livello	Descrizione
Fluente – 3	Le persone a questo livello si muovono in modo flessibile da un approccio all'altro. In quanto sviluppatori di software, possono rivedere un metodo per adattarlo a una nuova situazione senza precedenti.
Distacco – 2	Le persone a questo livello sono competenti in un unico approccio e sono pronte ad apprendere alternative. Possono adattare un metodo a una nuova situazione precedente.
A seguire – 1	Le persone a questo livello seguono un unico approccio e sono confuse quando si confrontano con qualcosa di nuovo. Possono eseguire passaggi del metodo, come la composizione di un modello o l'esecuzione di test.

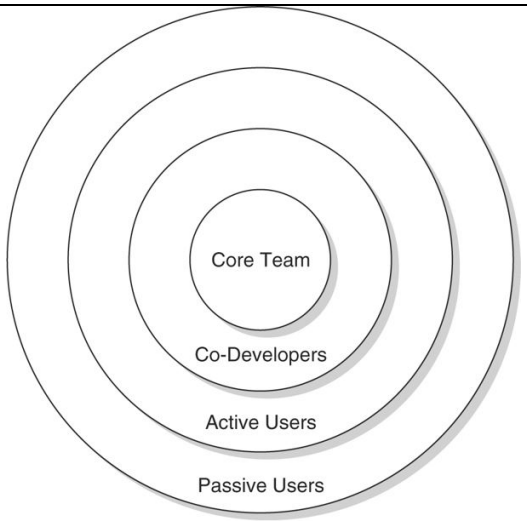


Figure 5.3 Struttura a cipolla di una comunità open source

Nella struttura a cipolla di una comunità open source, si distinguono quattro livelli di partecipazione:

- Il **core team** è costituito da un piccolo team di sviluppatori esperti che funge anche da team di gestione. Le modifiche ai componenti del kernel del software possono essere apportate solo dai membri del core team.
- I **co-sviluppatori** sono un gruppo più ampio di persone che circondano il core team. I co-sviluppatori esaminano il codice e apportano correzioni di bug.
- Gli **utenti attivi** sono gli utenti della versione più recente del sistema. Inviano segnalazioni di bug e richieste di funzionalità, ma non programmano essi stessi il sistema.
- Gli **utenti passivi utilizzano** semplicemente versioni stabili del software. Non interagiscono con gli sviluppatori.

Di solito, gli strati esterni contengono più persone rispetto agli strati interni. Spesso, il core team non contiene più di 5-15 persone. Ad esempio, (Mockus *et al.*, **2000**) riportano che il core team di 15 persone di Apache ha eseguito oltre l'80% della codifica delle funzionalità.

Questo tipo di organizzazione di progetti open source è una meritocrazia; vale a dire, le persone ricoprono ruoli basati sul talento e sulla qualità comprovata. Le persone in un livello possono passare al livello successivo più alto. Arrivare al centro si ottiene attraverso un processo di guadagno di fiducia, responsabilità e status attraverso la competizione e il talento dimostrato. Ad esempio, un utente attivo può diventare un co-sviluppatore suggerendo miglioramenti della qualità in un periodo di tempo. Allo stesso modo, un record di lunga data di correzioni di qualità al codice può far guadagnare a un co-sviluppatore una posizione nel team principale.

Il carattere volontario dello sviluppo open source pone alcune sfide specifiche:

- Motivazione a rimanere attivi
- Disaccordo tra gli sviluppatori
- Comunicazione tra sviluppatori

Una comunità open source è "un'azienda senza muri" (Fang e Neufeld, **2006**). Le persone possono entrare e uscire liberamente dalla comunità. Gli sviluppatori che partecipano a progetti open source raramente lo fanno disinteressatamente. Si aspettano qualcosa in cambio, come la capacità di imparare cose nuove, uno status più elevato all'interno del loro normale lavoro, attenzione perché fanno parte di un progetto di successo e così via. Questo non dovrebbe sorprenderti. I professionisti del software hanno esigenze di crescita elevate. I progetti open source che mettono alla prova le capacità degli sviluppatori, hanno una base di codice ben modulata e fanno uso di strumenti avanzati hanno maggiori possibilità di attrarre una comunità sostenibile.

Una delle cose peggiori che possono accadere a un progetto open source è il disaccordo tra gli sviluppatori. Un ostacolo comune nei progetti open source è il disaccordo sulla velocità di sviluppo (Godfrey e Tu, **2000**). Alcuni sviluppatori potrebbero voler pubblicare frequentemente nuove versioni, mentre altri potrebbero assumere una posizione più cauta. Un'altra potenziale fonte di disaccordo è quando gli utenti iniziano a sentirsi a disagio nella "democrazia non democratica" di un progetto open source. Sebbene molte persone possano inviare correzioni di bug o richieste di funzionalità, il potere di decidere cosa accade effettivamente di solito spetta a una o poche persone nel team principale. Se le richieste di uno sviluppatore vengono rifiutate più e più volte, potrebbe sentirsi frustrato e lasciare la comunità o decidere di creare un fork (prendere una copia del codice sorgente e avviare un percorso di sviluppo indipendente).

La comunicazione tra gli sviluppatori è un problema in ogni team distribuito. Ma nei progetti open source, la situazione è peggiore a causa dell'appartenenza alla comunità fluttuante e della mancanza di documentazione formale. Una chiara modularizzazione del codice è un

modo importante per ridurre la necessità di una comunicazione estesa. Le comunità open source tendono inoltre a utilizzare strumenti di controllo della configurazione, sistemi di tracciamento dei bug e mailing list per la comunicazione.

5.2.7 Principi generali per l'organizzazione di una squadra

Non importa come cerchiamo di organizzare una squadra, il punto chiave è che dovrebbe essere una *squadra.* Da molti test riguardanti la produttività nei progetti di sviluppo software, risulta più e più volte che i fattori riguardanti le capacità del team hanno un'influenza molto maggiore di qualsiasi altra cosa. Fattori come il morale, le norme di gruppo e lo stile di gestione svolgono un ruolo più importante di cose come l'uso di linguaggi di alto livello, la complessità del prodotto e così via (vedi, ad esempio, (Lawrence, **1981**)).

Alcuni principi generali per l'organizzazione del team sono riportati in (Koontz e O'Donnell, **1972**). In particolare, questi principi generali si applicano anche all'organizzazione di progetti di sviluppo software:

- **Usa meno persone, e meglio,** La massima produttività è raggiunta da un gruppo relativamente piccolo di persone. Questo vale per romanzieri, calciatori e muratori. Non c'è motivo di credere che non si applichi ugualmente ai programmatori e ad altre persone che lavorano nel campo del software. Inoltre, i grandi gruppi richiedono più comunicazione, che ha un effetto negativo sulla produttività e porta a più errori.
- **Cerca di adattare i compiti alle capacità e alla motivazione delle persone disponibili** In altre parole, fai attenzione che il Principio di Peter non si applichi alla tua situazione. In molte organizzazioni, programmatori eccellenti possono essere promossi solo in posizioni manageriali. È molto meglio offrire possibilità di carriera nelle aree più tecniche dello sviluppo e della manutenzione del software.
- **Aiuta le persone a ottenere il massimo da se stesse** Non dovresti perseguire nessuna delle seguenti situazioni:
    - – Il principio di Peter inverso, in cui le persone all'interno di un'organizzazione si elevano al livello in cui diventano indispensabili. Ad esempio, un programmatore può diventare l'unico esperto di un determinato sistema. Se non ha la possibilità di lavorare su nient'altro, non è improbabile che questa persona, in mancanza di un compito più interessante e stimolante, lasci la tua organizzazione. A quel punto sei davvero nei guai.
    - – Il principio di Paul, in cui le persone all'interno di un'organizzazione salgono al livello in cui la loro esperienza diventa obsoleta entro cinque anni. Data la velocità con cui i nuovi sviluppi entrano nel mercato dell'ingegneria del software e dell'informatica in generale, è molto importante che le persone abbiano l'opportunità di crescere e rimanere al passo con i nuovi sviluppi.
- **Selezionare le persone per garantire un team equilibrato e armonioso** In generale, questo significa che non è sufficiente avere solo pochi massimi esperti. Una squadra di calcio ha bisogno di giocatori regolari oltre che di stelle. Selezionare il giusto mix di

persone è un compito complicato. Ci sono vari buoni testi disponibili che affrontano specificamente questa domanda (per esempio, (Weinberg, 1971) e (Metzger, 1987)).

- **Rimuovere le persone che non si adattano alla squadra** Se si scopre che una squadra non funziona come un'unità coerente, siamo spesso inclini ad aspettare un po', vedere come si evolvono le cose e sperare che arrivino tempi migliori. A lungo termine, questo è dannoso.

5.3 RIEPILOGO

Il software è scritto dagli esseri umani. La loro produttività è in parte determinata da fattori quali il linguaggio di programmazione utilizzato, la velocità della macchina e gli strumenti disponibili. Tuttavia, l'ambiente organizzativo in cui operano è altrettanto importante. Una buona gestione del team si distingue da una cattiva gestione del team soprattutto per il grado di attenzione che viene prestata a questi fattori umani. L'elemento umano nella gestione dei progetti software è stato discusso nella **Sezione 5.1**, insieme a note tassonomie di meccanismi di coordinamento e stili di gestione.

Esistono diversi modi per organizzare gli sviluppatori di software in un team. Queste forme organizzative e alcune delle relative avvertenze sono state discusse nella **Sezione 5.2**. Le organizzazioni gerarchiche e a matrice non sono specifiche per lo sviluppo del software, mentre il capo programmatore, SWAT e i team agili sono nati nel campo del software. Ognuno di questi ultimi cerca in qualche modo di conciliare i due tipi di gestione tipicamente richiesti nei progetti di sviluppo software: un approccio individualistico e personale in cui si cerca di ottenere il meglio dai membri del team e uno stile di gestione gerarchico dall'alto verso il basso per portare a termine le cose nei tempi e nei limiti del budget. I progetti open source di successo di solito hanno un'organizzazione a forma di cipolla.

5.4 ULTERIORI LETTURE

Una fonte di informazioni ancora molto rilevante sui fattori psicologici relativi allo sviluppo del software è (Weinberg, **1971**). Anche Brooks (**1995**) e DeMarco e Lister (**1999**) contengono una serie di preziose osservazioni. I problemi di coordinamento nello sviluppo del software sono discussi in (Kraut e Streeter, **1995**). (Software, **1996a**) e (CACM, **1993b**) sono numeri speciali di riviste sulla gestione dei progetti software.

Mintzberg (**1983**) è il classico testo sull'organizzazione del management. Gli stili di gestione di base discussi nella **Sezione 5.1.2** sono basati su (Reddin, **1970**) e (Constantine, **1993**).

(Couger e Zawacki, **1980**) è una delle prime fonti sulle esigenze di crescita degli sviluppatori di software. (Procaccino *et al.*, **2006**) affrontano anche questo problema. Armor (**2002**) sottolinea l'importanza degli aspetti sociali e di comunicazione nei progetti di sviluppo del software.

Il capo del team di programmatori è descritto in (Baker, 1972). La sua forma modificata è descritta in (Macro e Buxton, 1987). SWAT è discusso in (Martin, 1991). I team agili sono descritti, tra gli altri, (Highsmith, 2004). I tre livelli di comprensione sono discussi in (Cockburn, 2002). (Software, 2005) contiene una serie di articoli su come adottare metodi agili.

(SPIP, 2006) contiene una raccolta di articoli sui processi di sviluppo open source. Crowston e Howison (2006) discutono della salute delle comunità open source. Aberdour (2007) discute i modi per ottenere la qualità nello sviluppo di software open source.

Esercizi

1. Spiegare la classificazione di Mintzberg delle configurazioni organizzative ei relativi meccanismi di coordinamento.

2. Discutere gli stili di gestione di base di Reddin.

3. Quali sono le criticità in un'organizzazione gerarchica del team?

4. Evidenzia le differenze tra un team di capo programmatori, un team SWAT e un team agile.

5. Quale degli stili di gestione di Reddin si adatta meglio a un team agile?

6. Qual è il Principio di Pietro? Dove emerge nello sviluppo del software?

7. Perché un team agile dovrebbe avere bisogno di persone migliori di un team che segue un approccio basato sulla pianificazione?

8. ♡Considera un progetto di sviluppo software in cui sei stato coinvolto. Quale stile di meccanismo di coordinamento o stile di gestione si adatta meglio a questo progetto? Ritenete che la gestione sia stata adeguata o la discussione nella **Sezione 5.1** indica possibili miglioramenti?

9. ♡Da un punto di vista gestionale, discuti dei possibili pro e contro dell'avere un assistente tecnico nel tuo team di sviluppo.

10. ♠Scrivi un saggio sul ruolo dei problemi delle persone nello sviluppo del software. Per fare ciò, puoi consultare alcuni dei libri che si concentrano sui problemi delle persone nello sviluppo del software, come (Brooks, 1995), (Weinberg, 1971) e (DeMarco e Lister, 1999).

11. ♠Discutere i pro ei contro di un'organizzazione in cui la dipartimentalizzazione primaria è verticale (cioè per specialità, come database, interfacce uomo-computer e programmazione grafica) rispetto a quella in cui la dipartimentalizzazione primaria è orizzontale (cioè per fase, come progettazione, implementazione e test).

12. ♡Scrivi un saggio su come vengono gestiti i progetti di sviluppo di software open source.

13. ♡Discutere i pro ei contro di lasciare che le persone ruotino tra progetti di diversi domini applicativi invece di farli diventare veri esperti in un particolare dominio applicativo.

Un esempio lampante è nel seguente aneddoto tratto da (Weinberg, **1971**). Il direttore di un centro informatico universitario ha ricevuto lamentele per studenti e programmatori che chiacchieravano e ridevano davanti alla macchinetta del caffè del dipartimento. Essendo un vero manager, preoccupato per la produttività, ha spostato la macchina del caffè in un posto remoto. Subito dopo, il carico sui consulenti del centro di calcolo è aumentato considerevolmente. La folla vicino alla macchina del caffè era infatti un canale di comunicazione efficace e informale, attraverso il quale la maggior parte dei problemi venivano risolti.